DATA ENGINEERING MASTER PREP • EYE-CARE WARM PARCHMENT EDITION

Data Engineering — Complete 3-Pillar Theory Textbook

All 3 Pillars in One Volume: Storage Engines, SQL & Warehousing • Distributed Compute, Apache Spark & Kafka • Lakehouse (Iceberg/Delta), Streaming & Orchestration

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

25 Sections Indexed

Pillar 1: Storage Engines, Columnar Formats & Analytical SQL — Master Textbook ..... p. 2

Day 1: Row vs Columnar (Parquet), B-Tree vs LSM-Tree ..... p. 2

1.1 Storage Engine Internals: B+Tree (OLTP) vs LSM-Tree (Write-Heavy) (DE01) ..... p. 2

1.2 Apache Parquet Architecture: PAX Layout, Pushdowns & Encodings (DE02) ..... p. 3

1.3 Dremel / Parquet Striping: Repetition Level (R) & Definition Level (D) (DE03) ..... p. 4

Day 2: Dimensional Modeling (SCDs) & Analytical SQL ..... p. 4

2.1 Kimball Dimensional Modeling & Slowly Changing Dimensions (DE04) ..... p. 4

2.2 Analytical SQL Traps: Window Frames & Three-Valued NULL Logic (DE05 , DE06) ..... p. 4

Pillar 2: Distributed Systems, Apache Spark & Kafka — First-Principles Guide ..... p. 6

Day 3: Apache Spark Execution, Joins & Skew Optimization ..... p. 6

1. Spark Execution Hierarchy: Jobs, Stages & Tasks (DE07) ..... p. 6

2. Physical Join Strategies in Spark SQL (DE08) ..... p. 7

3. Diagnosing & Fixing Partition Data Skew (DE09) ..... p. 7

Day 4: Apache Kafka Internals, Replication & EOS ..... p. 8

1. Partitions, Consumer Groups & Parallelism Invariants (DE10) ..... p. 8

2. Broker Replication, ISR & The Zero-Data-Loss Triad (DE11) ..... p. 9

3. Delivery Guarantees & End-to-End Exactly-Once Semantics (DE12) ..... p. 9

Pillar 3: Lakehouse (Iceberg/Delta), Streaming & Orchestration — First-Principles Guide ..... p. 11

Day 5: Stream Processing, Watermarks & Stateful Windows ..... p. 11

1. Event Time vs. Processing Time & Watermarks (DE13) ..... p. 11

2. Streaming Window Geometries (DE14) ..... p. 12

Day 6: Data Lakehouse (Iceberg/Delta), Medallion & Orchestration ..... p. 12

1. Open Table Formats: Apache Iceberg & Delta Lake (DE15) ..... p. 12

2. Small File Compaction, Partitioning & Z-Ordering (DE16) ..... p. 13

3. Airflow Idempotency & The Medallion + WAP Architecture (DE17 , DE18) ..... p. 14

Pillar 1: Storage Engines, Columnar Formats & Analytical SQL — Master Textbook

↑ Mục Lục

> 💡 **TIP**
>
> **How to Use This Guide:** Every section below maps directly to your `DE01` – `DE06` Data Engineering questions. Use the **Contents** menu to jump to any sub-topic, or enable 📄 **Textbook: on** in 🤖 **Ask AI** so the AI tutor cites these exact derivations and tables.

Day 1: Row vs Columnar (Parquet), B-Tree vs LSM-Tree

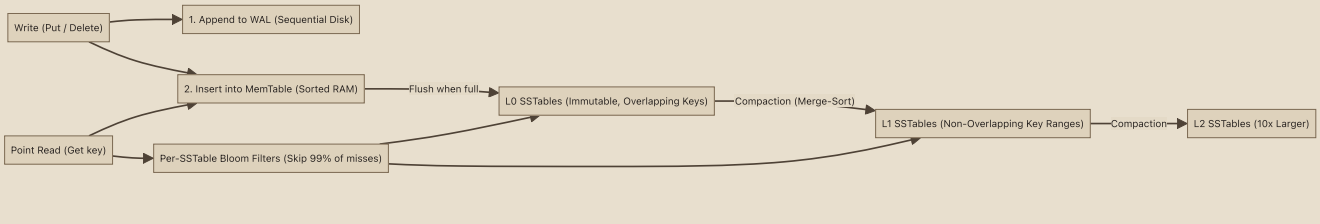
↑ Mục Lục

1.1 Storage Engine Internals: B+Tree (OLTP) vs LSM-Tree (Write-Heavy) (`DE01`)

💡 Intuition (Plain English First)

Every database storage engine optimizes a fundamental trade-off between **Read Amplification**, **Write Amplification**, and **Space Amplification** (the **RUM Conjecture**):

1. **B+Tree (PostgreSQL, MySQL InnoDB, Oracle):**
   - Stores data in fixed-size mutable disk pages (8–16 KB) organized as a balanced B-ary search tree where all rows live in the leaf pages.
   - **Strengths:** Point lookups and range scans touch only $\mathcal{O}(\log_B N)$ pages (2–4 disk seeks).
   - **Weakness:** Updating a single 100-byte row on a random key requires reading an entire 16 KB page, modifying it, and flushing the 16 KB page back to disk (**random I/O + high write amplification**).
2. **Log-Structured Merge-Tree (LSM-Tree — RocksDB, Apache Cassandra, HBase, ScyllaDB):**
   - **Never modifies disk files in place!** Every insert, update, or delete (`tombstone`) is appended sequentially to an on-disk **Write-Ahead Log (WAL)** and inserted into a sorted in-memory buffer (**MemTable**, typically a SkipList or B-Tree).
   - When the MemTable reaches capacity (e.g., 64 MB), it is flushed sequentially to disk as an **immutable Sorted String Table (`SSTable`)** at Level L_0.
   - Background **Compaction** threads periodically merge-sort overlapping SSTables from $L_k \rightarrow L_{k+1}$, discarding overwritten values and expired tombstones.



📐 Side-by-Side Comparison (`DE01`)

Metric / Property	B+Tree (`InnoDB` , `PostgreSQL`)	LSM-Tree (`RocksDB` , `Cassandra` , `HBase`)
Write I/O Pattern	**Random I/O** (in-place 8–16 KB page overwrites + page splits)	**Sequential I/O** (append-only WAL + immutable SSTable flushes)
Point Read Latency	**Fast & predictable** ($\mathcal{O}(\log_B N)$, 1 leaf page)	**Higher tail latency** (must check MemTable $+L_0 + L_1 \ldots$; mitigated by **Bloom filters**)
Delete Mechanism	Removes entry directly from leaf page	Writes a **Tombstone** marker; space is reclaimed only during **Compaction**
Best Workload	Read-heavy / transactional **OLTP**	Write-intensive event ingestion, time-series, wide-column stores

---

1.2 Apache Parquet Architecture: PAX Layout, Pushdowns & Encodings (`DE02`)

💡 Intuition (Plain English First)

Why is **Apache Parquet** the universal storage format for modern Data Lakes and Lakehouses? Because it uses the **PAX (Partition Attributes Across)** hybrid layout:

1. **Horizontal Slicing into Row Groups (**$\sim$ 128 MB–1 GB**):** The table is first partitioned into large horizontal batches of rows (e.g., 10^6 rows per Row Group) so distributed workers (Spark executors) can process different Row Groups in parallel.
2. **Vertical Slicing into Column Chunks:** Inside each Row Group, all values of Column A are stored contiguously as a **Column Chunk**, followed by all values of Column B, and so on.
3. **Pages (**1 MB**):** Each Column Chunk is subdivided into **Dictionary Pages** and **Data Pages** with per-page `min` / `max` statistics.
4. **File Footer at the End of the File:** Contains the schema and every Row Group's Column Chunk byte offsets, `min` , `max` , and `null_count` .
   - *Why is the metadata in the **footer** instead of the header?* Because Parquet files are written in a single streaming pass! You don't know the byte offsets or `min` / `max` statistics of all Row Groups until the final row has been written!

📐 The Two I/O Pruning Mechanisms (`DE02`)

1. **Projection Pushdown (Column Pruning):** If a table has $C = 50$ columns and your query references $k = 3$ columns, the reader reads the footer first and issues byte-range reads for **only those** 3 **Column Chunks**, eliminating $\frac{50-3}{50} = 94\%$ of column I/O.
2. **Predicate Pushdown (Row Group / Page Skipping):** For a filter `WHERE event_date = '2026-11-01'` , the reader checks each Row Group's `[min, max]` in the footer:

> $$\text{Skip Row Group } i \iff \text{'2026-11-01'} < \min_i(\text{event_date}) \ \vee \ \text{'2026-11-01'} > \max_i(\text{event_date})$$

> ⚠ **WARNING**
>
> **Exam Trap:** If `event_date` is unsorted (randomly distributed across all Row Groups), then **every** Row Group has min ≈ earliest date and max ≈ latest date, so `min` / `max` pruning skips 0 **Row Groups**! Always **sort** or **Z-Order** Parquet files by your high-selectivity filter columns.

1.3 Dremel / Parquet Striping: Repetition Level (R) & Definition Level (D) (`DE03`)

📐 Exact Rules for Nested Schemas (`DE03`)

Given a nested path from root to leaf `a.b.c.leaf`:

- **Maximum Repetition Level ($R_{\max}$):** Equals the number of `repeated` fields along the path (`0` if there are no arrays/lists).
  - For each emitted value, $R \in [0, R_{\max}]$ tells the reader **at which repeated ancestor depth the list repeated** compared to the previous value ($R = 0$ means a brand-new top-level record).
- **Maximum Definition Level ($D_{\max}$):** Equals the number of `repeated` + `optional` fields along the path (`required` fields contribute `0` because they can never be missing!).
  - When a leaf value is non-null, $D = D_{\max}$. When a value is `NULL`, $D < D_{\max}$ tells the reader **how many optional/repeated ancestors were defined** before the `NULL` occurred.

Day 2: Dimensional Modeling (SCDs) & Analytical SQL

↑ Mục Lục

2.1 Kimball Dimensional Modeling & Slowly Changing Dimensions (`DE04`)

📐 Complete SCD Reference Table (`DE04`)

SCD Type	Mechanism	History Preserved?	Storage / Join Rule
Type 0	Retain original attribute value forever	Original only	Never updated (e.g., `signup_date`)
Type 1	**Overwrite** attribute in place	**No** (destroys history)	Historical facts retroactively switch to the new attribute
Type 2	**Add a new row** with a new **Surrogate Key** (`_sk`), `(valid_from, valid_to, is_current)`	**Yes (Unlimited)**	Fact table **must join on Surrogate Key** (`_sk`) (or natural key + timestamp between `valid_from` and `valid_to`)
Type 3	**Add a new column** (`prev_city`, `effective_date`)	**Only 1 prior state**	Cannot attribute historical facts across multiple changes

---

2.2 Analytical SQL Traps: Window Frames & Three-Valued `NULL` Logic (`DE05`, `DE06`)

1. The Window Frame Default Trap (`DE05`)

Whenever you write `<func>() OVER (PARTITION BY p ORDER BY o)` **without** an explicit `ROWS` or `RANGE` frame:

- SQL defaults to:

  > `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`

- **Trap 1 (`LAST_VALUE`):** Because the window ends at `CURRENT ROW`, `LAST_VALUE(x) OVER (PARTITION BY dept ORDER BY dt)` returns the **current row's** `x`! Always write:

```sql
LAST_VALUE(salary) OVER (
  PARTITION BY dept ORDER BY hire_date
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
```

- **Trap 2 (`RANGE` vs `ROWS` on duplicate `ORDER BY` values):** `RANGE` treats all rows with equal `ORDER BY` values as a single peer group and sums them together on the first tied row. Use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` when you want a strict row-by-row running sum.

2. The `NOT IN` + `NULL` Three-Valued Logic Trap (`DE06`)

In SQL, `x NOT IN (a, b, NULL)` is parsed as:

> $$(x \neq a) \land (x \neq b) \land \underbrace{(x \neq \texttt{NULL})}_{\texttt{UNKNOWN}} \implies \texttt{UNKNOWN}$$

Because `WHERE` filters out both `FALSE` and `UNKNOWN`, **a single `NULL` inside a `NOT IN` subquery causes the outer query to return** 0 **rows**!

- **Safe Idioms:** Always use `WHERE NOT EXISTS (SELECT 1 FROM sub WHERE sub.id = main.id)` or `LEFT ANTI JOIN`.

Pillar 2: Distributed Systems, Apache Spark & Kafka — First-Principles Guide

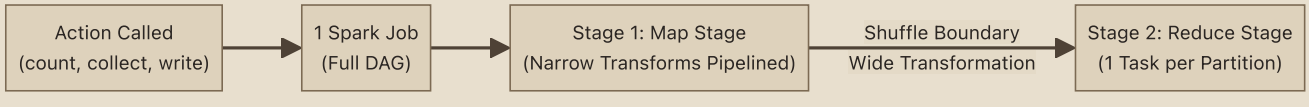
↑ Mục Lục

Day 3: Apache Spark Execution, Joins & Skew Optimization

↑ Mục Lục

1. Spark Execution Hierarchy: Jobs, Stages & Tasks (`DE07`)

Apache Spark's Catalyst Optimizer and DAG Scheduler compile high-level DataFrame / SQL queries into an execution plan:



- **Lazy Evaluation:** Transformations (`select` , `filter` , `join` , `groupBy`) only build the logical/physical plan. Nothing executes until an **Action** (`count()` , `collect()` , `show()` , `write.parquet()`) is invoked.
- **Narrow vs. Wide Transformations:**
  - **Narrow Transformation (0 Network Shuffle):** Each output partition is computed from at most one input partition (or colocated partitions on the same executor). Examples: `select` , `filter` , `withColumn` , `mapPartitions` , `union` , `coalesce(smaller_N, shuffle=False)` , and **Broadcast Hash Join**. Consecutive narrow transformations are fused into a single in-memory pipeline (**Whole-Stage CodeGen**) within **the same Stage**.
  - **Wide Transformation (Shuffle = Stage Boundary):** Output partitions require records hashed/sorted from *all* parent partitions across the network. Examples: `groupBy().agg()` , `distinct()` , `repartition()` , `orderBy()` / `sort()` , and **Sort-Merge Join**. Every wide transformation introduces a **Shuffle Boundary** and starts a **new Stage**.

> ⚠️ **WARNING**
>
> **The `coalesce(1)` Upstream Parallelism Trap:** Because `coalesce(1)` (`shuffle=False`) is a Narrow transformation, placing `df.filter(...).coalesce(1)` without a shuffle boundary forces Spark to collapse the **entire Stage's upstream task count to 1 single task**, reading and filtering all input files on a single CPU core! Use `repartition(1)` if you want upstream filtering to run across 200 parallel tasks before merging into 1 output file.

2. Physical Join Strategies in Spark SQL (`DE08`)

Join Strategy	Shuffle Required?	Sort Required?	Best Use Case & Constraints
Broadcast Hash Join (`BHJ`)	**No** (0 B of large table)	No	One table fits in Driver & Executor RAM ($<$ 10–100 MB, `autoBroadcastJoinThreshold`). Supports all joins **except** `FULL OUTER` and broadcasting the preserved side of an outer join (e.g., cannot broadcast left table in `LEFT JOIN`).
Sort-Merge Join (`SMJ`)	**Yes (Both tables)**	**Yes ($O(N \log N)$)**	Default scalable join for two large tables (`500 GB` $\bowtie$ `200 GB`). Spills gracefully to disk if partitions exceed memory. Equi-joins only (`=`).
Shuffle Hash Join (`SHJ`)	**Yes (Both tables)**	No	Shuffles both tables by key, then builds an in-memory hash table of the smaller partition per task without sorting. Faster than SMJ when one side is 3$\times$ smaller per partition and fits in memory.
Broadcast Nested Loop (`BNLJ`)	No	No	Fallback for **non-equi joins** (`a.ts BETWEEN b.start AND b.end`). $O(N \cdot M)$ complexity — dangerous on large tables!

3. Diagnosing & Fixing Partition Data Skew (`DE09`)

The Symptom: `199 / 200` Tasks Finished, `1` Straggler Stuck

When Spark performs a wide `groupBy(key)` or `join(key)`, it partitions records by:

$$\text{partition_id} = \text{hash}(\text{key}) \bmod P$$

If 40% of all rows share a single hot key (e.g., `merchant_id = NULL` or `'AMAZON'`), **every single one of those rows hashes to the exact same partition**, regardless of whether `spark.sql.shuffle.partitions` (P) is `200` or `20,000`!

Three Production Fixes for Hot-Key Skew

1. **Filter or Isolate `NULL` / Default Keys Before the Shuffle:**
   - In an `INNER JOIN`, `NULL = NULL` never matches anyway — filter `WHERE key IS NOT NULL` before joining!
   - In a `LEFT JOIN`, split `df_null = df.filter(col("key").isNull())` and `df_valid = df.filter(col("key").isNotNull())`, join only `df_valid`, and `unionByName` the two branches.
2. **Spark 3+ Adaptive Query Execution (AQE) Skew Join:**
   - Enable `spark.sql.adaptive.enabled = true` and `spark.sql.adaptive.skewJoin.enabled = true`.
   - At the end of the map stage, Spark inspects the actual byte sizes of shuffle files on disk. If Partition k exceeds `skewedPartitionFactor` (5$\times$ median) and `skewedPartitionThresholdInBytes` (256 MB), AQE splits Partition k into S sub-partitions and duplicates the matching partition from the other table.
3. **Manual Key Salting (K Buckets):**
   - **Fact / Skewed Table:** Append a uniform random integer $s \in \{0, 1, \ldots, K-1\}$ to the join key:

$$\text{salted_key} = \text{concat}(\text{key}, \text{'_'}, \lfloor \text{rand}() \times K \rfloor)$$

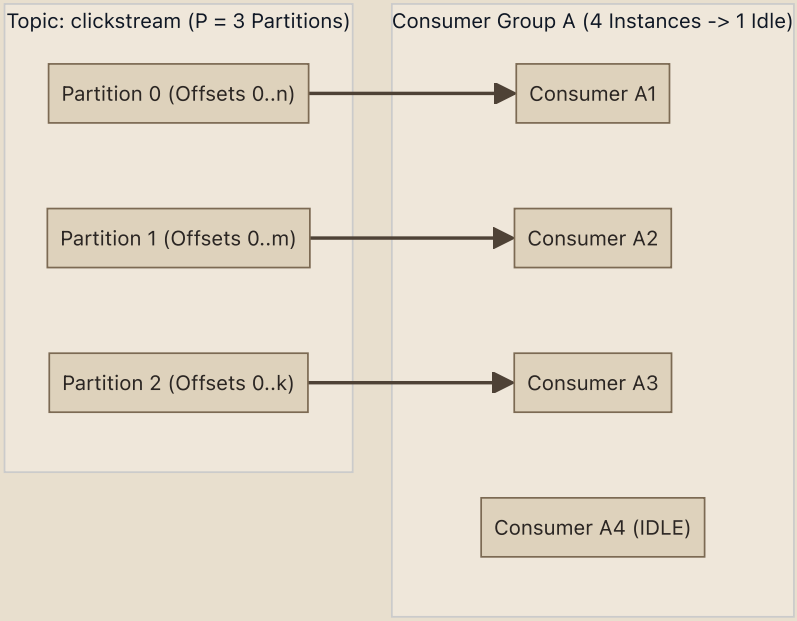
- **Dimension / Lookup Table:** Replicate each hot key K times using `explode(array(0, 1, ..., K-1))` so every salted bucket $\{0, \ldots, K-1\}$ finds its matching dimension row across K parallel tasks!

Day 4: Apache Kafka Internals, Replication & EOS

↑ Mục Lục

1. Partitions, Consumer Groups & Parallelism Invariants (`DE10`)

An Apache Kafka **Topic** is divided into P append-only, immutable log files called **Partitions**.



1. **Key Routing & Ordering Guarantee:**
   - When a producer sends a record with a non-null `key`, Kafka routes it deterministically via:

     $$\text{partition} = \text{murmur2}(\text{key}) \bmod P$$

   - **Ordering is strictly guaranteed ONLY within a single partition** (by offset), never globally across the whole topic.
2. **Consumer Group Exclusive Assignment Rule:**
   - Within a single **Consumer Group**, each partition is assigned to **at most one** consumer instance at any time.
   - If a Consumer Group has C consumers and the topic has P partitions:
     - If $C \leq P$: All C consumers are active (some read multiple partitions).
     - If $C > P$: Exactly P consumers read 1 partition each, and $C - P$ **consumers sit completely idle** as hot standbys.

- Multiple independent **Consumer Groups** (e.g., `realtime-fraud` and `s3-archiver`) each receive the full stream independently without interfering with each other.

---

2. Broker Replication, ISR & The Zero-Data-Loss Triad (`DE11`)

To prevent data loss when a broker crashes while maintaining high availability for writes, production Kafka clusters rely on four coupled parameters:

Parameter	Scope	Production Value	Why It Is Essential
`replication.factor` (`RF`)	Topic / Broker	`3`	Stores 1 Leader replica + 2 Follower replicas across 3 distinct racks/brokers.
`min.insync.replicas`	Topic / Broker	`2`	Requires at least 2 replicas (Leader + ≥ 1 Follower) to be in the **In-Sync Replica (ISR)** set before accepting `acks=all` writes.
`acks`	Producer	`all` (`-1`)	Producer waits until the Leader **and all current ISR followers** have written the record before marking the send successful.
`unclean.leader.election.enable`	Topic / Broker	`false`	Forbids an out-of-sync replica (not in ISR) from becoming Leader if all ISR brokers go down, preventing silent truncation of committed messages.

> ⚡ **IMPORTANT**
>
> **Why `RF = 3` and `min.insync.replicas = 2` (Not `3`):**
>
> - With `acks = 1`, if the Leader acknowledges a write and crashes 1 ms before followers pull the record, that record is permanently lost.
> - With `RF = 3`, `min.insync.replicas = 3`, and `acks = all`, if **even** 1 **broker reboots** for a rolling patch, the ISR size drops to $2 < 3$, and every producer immediately fails with `NotEnoughReplicasException` (0% write availability during maintenance!).
> - Setting `RF = 3, min.insync.replicas = 2, acks = all` guarantees every committed message lives on **at least** 2 **brokers** while tolerating 1 **broker outage** with zero downtime and zero data loss.

---

3. Delivery Guarantees & End-to-End Exactly-Once Semantics (`DE12`)

When a consumer reads a batch from Kafka, processes it, and writes to a downstream sink, the order of operations determines what happens on a crash:

1. **At-Most-Once (Commit Offset *Before* Writing to Sink):**
   - If the worker commits offset `100` to `__consumer_offsets` and crashes *before* writing to the database, upon restart it resumes at offset `101`. Message `100` is **silently lost**.
2. **At-Least-Once (Commit Offset *After* Writing to Sink):**
   - If the worker writes message `100` to the database and crashes *before* committing offset `100`, upon restart it reads message `100` again and writes it a second time (**duplicates**).

3. **Exactly-Once Semantics (EOS):**
    - **Producer-to-Broker (`enable.idempotence = true`):** Assigns each producer a `PID` and monotonically increasing sequence number per partition so broker-level network retries never create duplicate log entries.
    - **Kafka-to-Kafka (`transactional.id` + `read_committed`):** Uses Kafka's two-phase transaction coordinator (`sendOffsetsToTransaction`) to atomically commit consumer offsets and output topic messages together.
    - **Kafka-to-External Database (PostgreSQL / Snowflake / Delta):** Combine **At-Least-Once** offset commits (commit *after* DB write) with **Idempotent Sink Writes** (`INSERT ... ON CONFLICT (event_id) DO UPDATE` / `MERGE`), **or** store the Kafka `(topic, partition, offset)` inside a metadata table within the **exact same atomic ACID database transaction** as the business rows!

Pillar 3: Lakehouse (Iceberg/Delta), Streaming & Orchestration — First-Principles Guide

↑ Mục Lục

Day 5: Stream Processing, Watermarks & Stateful Windows

↑ Mục Lục

1. Event Time vs. Processing Time & Watermarks (`DE13`)

In distributed stream processing (Apache Flink, Spark Structured Streaming), every event carries two distinct timestamps:

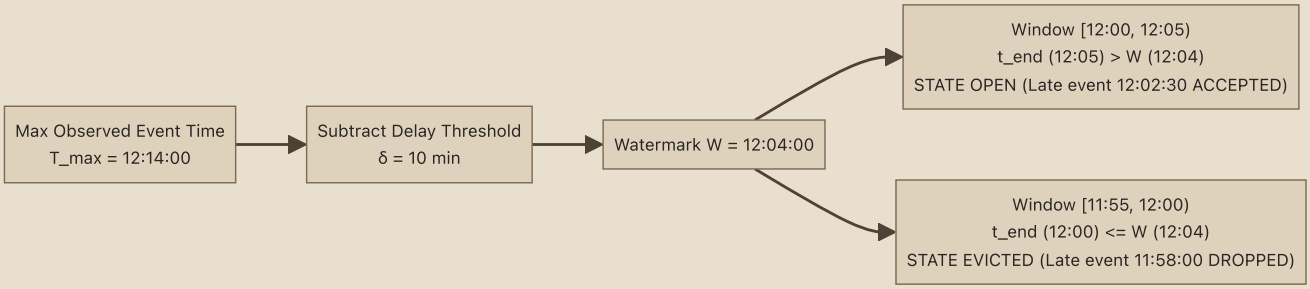
- **Event Time (t_{event}):** When the action physically occurred on the user's device (embedded inside the payload).
- **Processing Time (t_{proc}):** Wall-clock time of the server executing the streaming operator when the record is processed.

Why Watermarks Are Required for Event-Time Windows

Because mobile networks, retries, and Kafka partition lags delay records, events arrive **out of order**. If a streaming job computes 5-minute window aggregations `[12:00, 12:05)` by **Event Time**, how long should the engine keep that window's state in RocksDB/memory waiting for stragglers?

A **Watermark** $W(t)$ is a monotonically advancing timestamp threshold asserting that the stream is *complete* up to time $W(t)$:

$$W(t) = \max_{e \text{ seen so far}} \left(t_{\text{event}}(e) \right) - \delta_{\text{delay}}$$



- **Window Lifecycle Rule:** A window $[t_{\text{start}}, t_{\text{end}})$ stays open in state as long as $W < t_{\text{end}}$. As soon as $W \geq t_{\text{end}}$, the window emits its final result and its state is evicted (unless an explicit `allowedLateness` / side-output policy is configured).

2. Streaming Window Geometries (`DE14`)

Window Type	Geometry & Parameters	Windows per Event	Typical Engineering Use Case
Tumbling Window	Fixed size S, non-overlapping (`size = 5 min`).	**Exactly** 1	Minute/hourly revenue rollups (`[12:00, 12:05)` , `[12:05, 12:10)`).
Sliding (Hopping) Window	Fixed size S, advances every slide $L < S$ (`size = 10 min, slide = 2 min`).	$\frac{S}{L}$ (e.g., $10/2 = 5$)	Moving averages (e.g., 10-minute rolling error rate updated every 2 minutes).
Session Window	Dynamic variable length per key; closes after inactivity gap Δ_{gap} (`gap = 30 min`).	1 **merged session**	User browsing sessions, clickstream engagement funnels.

Day 6: Data Lakehouse (Iceberg/Delta), Medallion & Orchestration

↑ Mục Lục

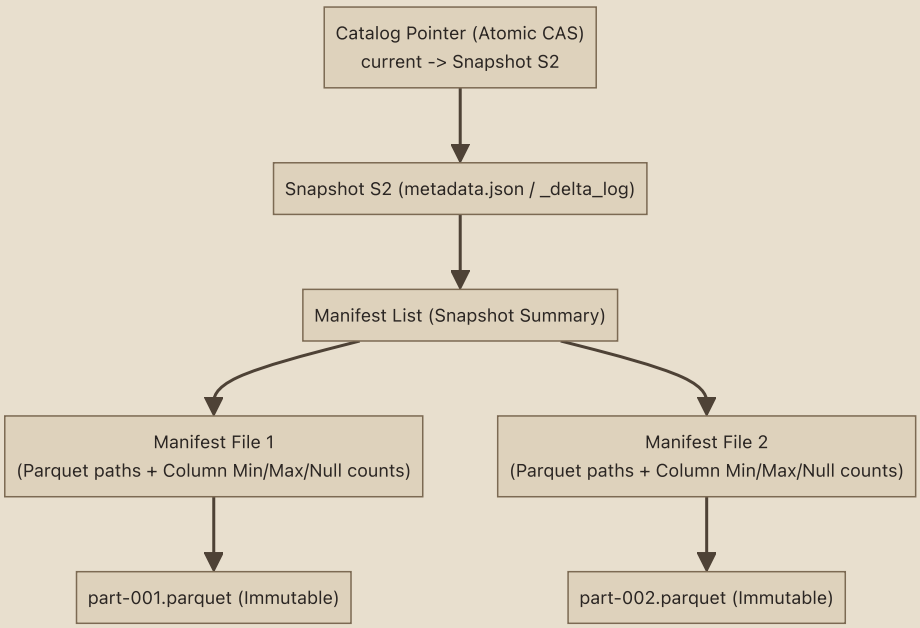
1. Open Table Formats: Apache Iceberg & Delta Lake (`DE15`)

Classic Hive-style Data Lakes on object storage (`s3://bucket/table/dt=2026-03-01/*.parquet`) suffer from three fatal flaws:

1. **No Atomic Multi-File Commits:** If a Spark job crashes after writing 40 out of 100 Parquet files, readers see corrupted partial data.
2. **Slow Directory Listing (`S3 LIST`):** Planning a query requires listing thousands of prefixes over HTTP.
3. **No Safe Concurrent Reads & Writes:** Mutating or compacting files while readers scan them causes `FileNotFoundException` .

How Iceberg & Delta Lake Deliver ACID on Immutable Parquet

Instead of using directory listings as the source of truth, **Apache Iceberg** and **Delta Lake** maintain a versioned **Metadata Tree / Transaction Log** above immutable Parquet files:



- **Snapshot Isolation & Time Travel:** Readers read a pinned snapshot (S_n) and never see uncommitted files. Querying `SELECT * FROM events VERSION AS OF 42` or `TIMESTAMP AS OF '2026-03-01 00:00:00'` simply reads an older snapshot pointer before `VACUUM` / `expire_snapshots` deletes unreferenced files.
- **Optimistic Concurrency Control (OCC):** Concurrent writers prepare Parquet files optimistically and attempt an atomic compare-and-swap on the catalog/log pointer. Non-conflicting partition writes automatically rebase and succeed; conflicting row-level writes retry or abort cleanly.

---

2. Small File Compaction, Partitioning & Z-Ordering (`DE16`)

1. **The Small File Problem & Compaction (`OPTIMIZE` / `rewrite_data_files`):**
   - High-frequency streaming sinks write thousands of 1 MB files per day, degrading query latency due to S3 metadata and HTTP connection overhead.
   - **Bin-Packing Compaction** periodically rewrites small files into optimal 128 MB–1 GB Parquet files in a new snapshot.
2. **Coarse Physical Partitioning vs. Multi-Dimensional Z-Ordering:**
   - **Physical Partitioning (`PARTITIONED BY (event_date)`):** Only partition by **low-cardinality** columns ($\leq 10^3$–10^4 partitions total) so each partition holds ≥ 1 GB. Partitioning by high-cardinality columns like `user_id` causes catastrophic **over-partitioning** (billions of tiny files).
   - **Why Linear `ORDER BY (colA, colB)` Fails on `colB`:** Linear sorting clusters `colA` tightly, but spreads `colB` across every file, resulting in 0% file skipping for queries filtering only on `colB`.
   - **Z-Ordering (`ZORDER BY (user_id, merchant_id)`):** Interleaves the bits of multiple columns along a **Morton space-filling curve**, preserving locality in 2D/3D space so Parquet file `[min, max]` statistics skip $> 90\%$ of files whether filtering on `user_id`, `merchant_id`, or both!

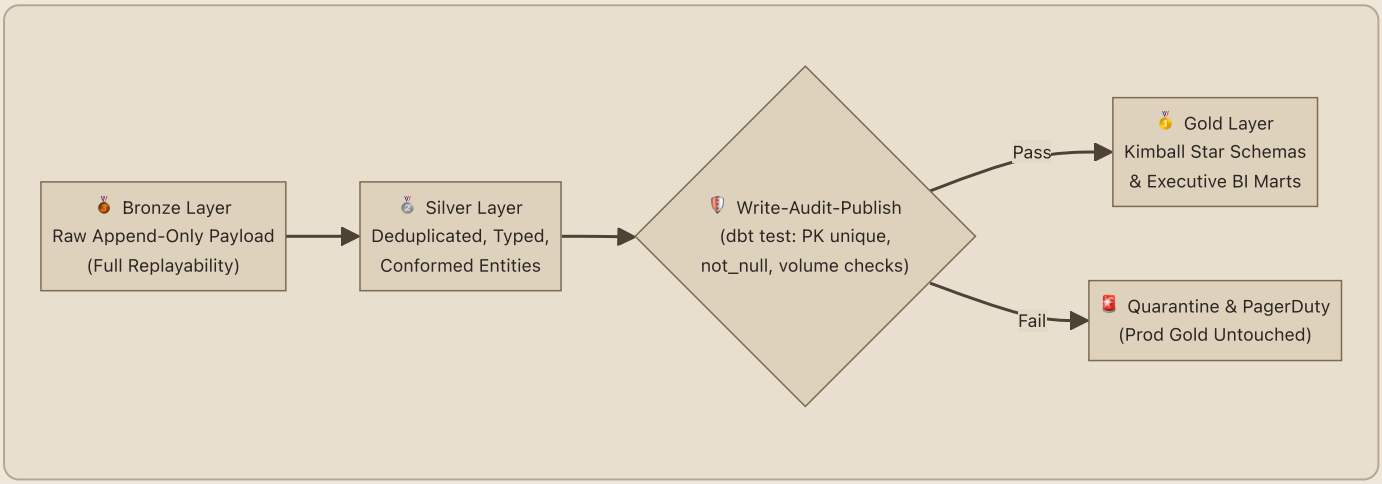
---

3. Airflow Idempotency & The Medallion + WAP Architecture (`DE17`, `DE18`)

Two Golden Rules of Idempotent Airflow DAGs (`DE17`)

1. **Deterministic Time Boundaries (`data_interval_start` / `{{ ds }}`):** Never use `NOW()` or `CURRENT_DATE()` inside SQL tasks. Always filter using Airflow's logical interval (`WHERE ts >= '{{ data_interval_start }}' AND ts < '{{ data_interval_end }}'`) so retries and historical **backfills** (`airflow dags backfill`) process the exact target window.
2. **Atomic Partition Overwrite (`INSERT OVERWRITE` / `MERGE`):** Never use blind `INSERT INTO` on retryable tasks. Overwrite only the target logical date partition (`INSERT OVERWRITE TABLE fct PARTITION (dt = '{{ ds }}')`) so running the task 1× or 10× yields the exact same row count.

Medallion Architecture & Write-Audit-Publish (`DE18`)



- **Bronze:** Raw, immutable, append-only source payloads + ingestion metadata (`_ingested_at`). Enables full historical replay if downstream parsing logic changes.
- **Silver:** Deduplicated (`ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1`), schema-enforced, conformed core entities.
- **Gold:** Business-ready Star Schemas (`fct_*`, `dim_*`), ML feature sets, and pre-aggregated BI marts.
- **Write-Audit-Publish (WAP):** Writes transformed records to an isolated staging table or catalog branch (`Write`), executes automated data quality contracts (`Audit` via `dbt test` / Great Expectations), and atomically swaps/merges into production (`Publish`) only if 100% of checks pass.